

Understanding null in Java

Understanding null in Java

Java and null share a unique bond. Almost all of the Java developers get trouble with the `NullPointerException` and this is the most infamous fact about null and Java. In this journal, we will be understanding null in Java.

Null was there for a long time and we all know that null creates more problems than it solves. So, let's try to learn more about the concepts of null and make sure that we use it correctly.

Many of the developers are already familiar with null but for those who are not, they can learn some old and new things about null keyword.

What is null?

null is a very important concept in Java. The original intention of inventing null was to denote the absence of something. But over the years it has troubled Java developers with the nasty `NullPointerException`.

Read More: [How to Handle Null Pointer Exception in Java](#)

Before we start, let's recall and understand what is variable and what is a value? Variables are basically the containers that store the value inside them. Every time we declare a variable, we need to specify the type that will define the kind of value which will be stored in the variable.

In Java, there are two major categories of types: primitive and reference. Variables that are declared of type primitive store values whereas variables declared as reference types store references. Given below is a basic example of variables and value assigned to those variables.

```
package in.developersjournal;

public class Main {

    // Uninitialised variable of reference type.

    // Hence it will store null in it.

    private static Object djObject;

    // Uninitialised int. It is of primitive type.

    // Hence it will store 0 until initialised.

    private static int firstInt;
```

Understanding null in Java

```
public static void main(String[] args) {  
    // Initialised integer with value 10.  
    int secondInt = 10;  
    System.out.println(secondInt);  
    System.out.println(firstInt);  
    System.out.println(djObject);  
}  
}
```

The above code snippet declares 3 variables:

1. Object djObject – Reference Type
2. int firstNumber – Primitive Type
3. int secondNumber – Primitive Type

Here the first two variables are not initialized while declaring them and the third variable is initialized with the number 10. Since the Object is of reference type thus it stores null in it and the firstNumber stores 0 as the default value.

Properties of null

#null Keyword / Reserved Word

In Java, null is a reserved word (keyword) for literal values. It seems like a keyword, but actually, it is a literal similar to true and false. The reserved word null is case sensitive and we cannot write null as Null or NULL, the compiler will not recognize them and give an error.

```
Object djObject1 = NULL; // Not Ok - Compile time error
```

```
Object djObject2 = null; // Ok
```

#null used as a default

Every primitive type of variable has a default value (e.g. int has 0, boolean has false) if not initialized at the time of declaration. Similarly null is the default value of any reference type

Understanding null in Java

variable which is not initialized at the time of declaration. This is true for all kinds of variables, instance variable or static variable, except that compiler will warn you if you use a local variable without initializing them.

#null – Casting to other types

null is not an Object or neither a type. It's just a special value, which can be assigned to any reference type. Typecasting null to any reference type is fine at both compile-time and runtime and it will not throw any error or exception.

```
// Typecasting null to String type
```

```
String myStr = (String) null;
```

```
// Typecasting to an Integer class
```

```
Integer myInt = (Integer) null;
```

```
// Typecasting to a Double class
```

```
Double myDb1 = (Double) null;
```

An important point to remember here is that null can only be assigned to reference types. We cannot assign null to primitive variables e.g. int, double, float or boolean. If we try to do so, then the compiler will complain.

```
// incompatible types : required int found null
```

```
int i = null;
```

```
// incompatible types : required short found null
```

```
short s = null;
```

```
// incompatible types : required byte found null
```

```
byte b = null;
```

#null – instanceof operator

instanceof operator will return false if used against any reference variable with null value or null literal itself. We can see it in action in code snippet below

```
public class NullInstanceOfExample {  
    public static void main(String[] args) {  
        Byte nullReferenceVariable = null;
```

Understanding null in Java

```
System.out.println("Checking instanceof with Byte type variable with null stored");  
if (nullReferenceVariable instanceof Byte) {  
    System.out.println("nullReferenceVariable is instance of Byte");  
} else {  
    System.out.println("nullReferenceVariable is NOT an instance of Byte");  
}  
}  
}
```

NullPointerException (NPE)

NullPointerException is a runtime exception. In Java, a special null value can be assigned to an object reference. NullPointerException is thrown when an application attempts to use an object reference with a null value.

Conclusion

That's about null in Java. Common scenarios around the null have been tried to cover: how to use it, where to use it, and how it can be used as the placeholder for reference type variables.

Always remember, null is the default value of any reference variable and you cannot call any instance method, or access an instance variable using null reference in Java.